AI Engineering and Claude Architect · Part V, Certification

Certification, Mock Exam 1

60 Items in 120 Minutes

Scenarios: Customer Support Resolution Agent, Code Generation with Claude Code, Multi-Agent Research System, Structured Data Extraction. Set a timer, work without notes, and mark your answers on paper.

Items	60, in 4 scenario sets of 15
Time	120 minutes
Domains	16, 11, 12, 12 and 9 items, rounded from the guide's weights
Status	Original practice items, written from the guide. Not real or leaked

Before You Start

Key idea

These are original practice items, not real or leaked exam questions, and the company names are invented. A score here doesn't predict your exam result.

Follow the real conditions: one sitting, no notes, flag and return. Where the guide and current products differ, the answers follow the guide, and an explanation may add a Build today note. The real exam's wording may be longer and harder, as candidates report.

Checkpoints: 28 minutes after Scenario 1, 56 after Scenario 2, 84 after Scenario 3 and 110 after Scenario 4.

Targets, which are my own and not official: 80% overall, and no domain under 70%.

Scenario: Customer Support Resolution Agent

Key idea

Harbor Outfitters is building a support resolution agent on the Claude Agent SDK. It has custom MCP tools `get_customer`, `lookup_order`, `process_refund` and `escalate_to_human`. The target is 80%+ first-contact resolution. Human agents see only what the agent sends them. Company policy allows the agent to refund up to $300.

Q1. During testing, a developer's loop sometimes ends with the customer's order unprocessed. The loop stops whenever a response contains a text block, on the theory that text means Claude has finished. What change fixes the cause?

A. Keep the text check, and add a regular expression that looks for phrases such as "let me check" before stopping
B. Stop only after a fixed number of iterations, chosen to be larger than the longest conversation in the logs
C. Decide on `stop_reason`, continuing on `tool_use` and stopping on `end_turn`, whether or not text is present
D. Add a system prompt rule that Claude must never write any text in a response that also calls a tool

Q2. Which design best fits this agent's mix of predictable and unpredictable requests?

A. Encode every return, billing and account path as a decision tree, and have Claude only write the wording of the replies
B. Fix one sequence of get_customer, lookup_order and process_refund, and run it for every incoming message
C. Let Claude choose all tools, and rely on an instruction asking it to follow the refund policy carefully
D. Let Claude choose the next tool from what it has learned so far, with code enforcing the few rules that must always hold

Q3. In 3% of logged cases, `process_refund` ran before identity had been verified, despite a clear instruction in the system prompt. Which change gives a deterministic guarantee?

A. Move the instruction to the top of the system prompt and repeat it in capital letters at the end as well
B. Block `process_refund` in code until `get_customer` has returned a verified customer ID in this session
C. Add a review step in which Claude confirms in text that it has verified identity before refunding
D. Add several few-shot examples that show verification happening before each refund is issued

Q4. A customer's message reads: "My jacket arrived torn, you charged me twice for it, and I've moved house." What should the agent do?

A. Resolve the torn jacket first, because it appears first, then ask the customer to send the other issues again
B. Treat these as three concerns, investigate them in parallel with shared context, and give one unified reply
C. Open three separate sessions so that each concern has a clean context, and send three separate replies
D. Escalate to a human, because any message with more than one concern is too complex for an agent to handle

Q5. A refund can't be completed, so the agent must hand over to a human. Which handoff best meets the requirements of this deployment? **Assume the human can't see the transcript.**

A. A structured summary with customer ID, root cause, refund amount and recommended action
B. The full conversation text, with every tool result, pasted into the ticket for completeness
C. A one-line note saying that the customer needs help with a refund and that a human should take over
D. A pointer to the conversation ID, so that the human can open the conversation in the agent's logs

Q6. `lookup_order` returns Unix timestamps, `get_customer` returns ISO 8601 dates, and status codes are numeric. Claude occasionally misreads them. What's the right place to fix this?

A. A PreToolUse hook that adds a preferred date format to each tool's outgoing arguments
B. A system prompt that lists the formats and asks Claude to convert them correctly on each turn
C. A PostToolUse hook that converts each result into one format before the model sees it
D. A new subagent that receives all tool results and rewrites them into a consistent format

Q7. A customer asks for a $900 refund. What's the best way to enforce the $300 limit, while still helping the customer?

A. Remove `process_refund` from the agent's tools, so that all refund requests of any size are sent to a human
B. Have `process_refund` silently cap the amount at $300, and tell the customer the refund has been completed
C. Add the limit to the system prompt along with examples of how to handle refunds over it, and rely on those
D. Intercept the `process_refund` call, block it, and return a reason that directs the agent to `escalate_to_human`

Q8. A manager wants the agent to run a fixed four-step chain for all requests, to make it easier to review. Which assessment is right?

A. A fixed chain is better for every agent, because reviews are the main goal of any production deployment
B. Fixed chains can't call MCP tools, so the agent must use dynamic decomposition for technical reasons alone
C. A fixed chain suits predictable multi-aspect work, while these high-ambiguity requests benefit from adaptive decomposition
D. A fixed chain would reduce context, so its first-contact resolution would be higher across every kind of request

Q9. `get_customer` is described as "Gets customer info", and `lookup_order` as "Gets order info". Claude often calls `get_customer` when asked about an order number. What's the most effective first fix?

A. Add a separate routing classifier that decides which tool to call before each model request is sent
B. Expand both descriptions with input formats, example queries and when to use each instead of the other
C. Add five few-shot examples of tool choice to the system prompt, and leave the tool descriptions as they are
D. Merge the two tools into one `lookup` tool that accepts any identifier and works out what it was given

Q10. When its database is down, `lookup_order` returns the text "Operation failed". The agent often apologizes and ends the conversation. What should the tool return?

A. An empty order list, so that the agent can continue without being interrupted by the failure
B. `isError` with a category such as transient, `isRetryable: true`, and a readable description
C. A longer message in the same format, written to ask the agent politely to try again a little later
D. An HTTP 500 status passed straight through as the tool's output, to stay close to the source system

Q11. `process_refund` rejects a refund because the 90-day return window has passed. What should the tool's response contain? **Select 2.**

A. A transient category, so that the agent tries again later
B. A customer-friendly explanation the agent can pass on
C. A generic error, in order to avoid exposing any policy details
D. An instruction telling the agent to retry using a lower amount
E. A flag showing that retrying won't help

Q12. The agent must always call `get_customer` before anything else on a new conversation. Which API setting ensures that the first call is that tool?

A. No `tool_choice` at all, while listing `get_customer` first in the tools array so it's considered first
B. `tool_choice: "auto"`, along with an instruction at the start of the system prompt to call it first
C. `tool_choice: "any"`, because this guarantees that the first response will call some tool or other
D. `tool_choice` naming the tool: `{"type": "tool", "name": "get_customer"}`, with later turns handled normally

Q13. The team plans to give this agent 18 tools, including inventory analytics and internal reporting. Why is that a concern?

A. More tools always make the context too large for the request, and the API will reject it with an error
B. Too many tools raise decision complexity and reduce selection reliability, so keep the tools the role needs
C. Every additional tool requires its own separate subagent, and this will increase latency for each request
D. MCP doesn't allow more than a small fixed number of tools to be connected for one agent at the same time

Q14. After a long conversation, the agent says "your refund of about a hundred dollars", although the customer's order was $127.40. Summarization had been used to save tokens. What's the best fix?

A. Increase `max_tokens`, so that the summaries Claude writes can be longer and include more of the details
B. Summarize more often, so that every summary is produced closer to the point where the amount was stated
C. Ask Claude in the system prompt to be careful with numbers whenever it writes its progressive summaries
D. Keep amounts, dates, order IDs and statuses in a persistent case facts block included with each prompt

Q15. A customer writes: "This is ridiculous. I need my delivery address changed." The agent can do this directly, and they haven't asked for a person. What should it do?

A. Ask the customer to rate their frustration, and escalate if the rating passes an agreed threshold

B. Acknowledge the frustration and make the change, escalating only if they then ask for a human

C. Escalate immediately, as negative sentiment is a strong signal that the case needs a human agent

D. Ignore the tone entirely and reply with the address change, avoiding any comment that might inflame things

Scenario: Code Generation with Claude Code

Key idea

Tessera Labs has 40 engineers using Claude Code in a monorepo with Terraform, Java services and a React frontend. They want consistent generation, refactoring and documentation, shared conventions, and clear guidance on planning.

Q16. A new engineer's Claude Code ignores the team's API naming standard. Everyone else follows it. The standard turns out to be in a senior engineer's `~/.claude/CLAUDE.md`, and others had copied it by hand. What's the right fix?

A. Ask the new engineer to paste the naming standard into the prompt at the start of each new session
B. Move it to the project's `CLAUDE.md` or `.claude/rules/`, so that it reaches everyone through version control
C. Add a step to the onboarding document telling each new engineer to copy that section to their own home directory
D. Share the senior engineer's whole `~/.claude/` directory on the company wiki for all engineers to download

Q17. Claude behaves differently for two engineers working in the same directory. What's the best first diagnostic?

A. Reinstall Claude Code for both engineers, and compare the behavior again from a clean installation
B. Ask Claude in each session to describe from recollection which conventions it thinks it is following
C. Run `/memory` in each session to see which memory files are loaded
D. Delete all `CLAUDE.md` files, and rebuild them one section at a time until the difference disappears

Q18. Each package has different standards, and a single root `CLAUDE.md` has grown to 2,000 lines. Which approaches does the guide support? **Select 2.**

A. Moving the content into each developer's `~/.claude/CLAUDE.md`
B. Splitting topics into focused files under `.claude/rules/`
C. Converting all of the standards into a single MCP tool for lookups
D. Using `@import` in each package's `CLAUDE.md` to pull in only the standards it needs
E. Keeping one file, while asking Claude to skip the irrelevant sections

Q19. The team wants a `/changelog` command that every engineer gets on clone. Where does it go, in the guide's terms?

A. `~/.claude/commands/` on each machine
B. The root `CLAUDE.md` file
C. `.claude/commands/` in the repository
D. A `.mcp.json` server entry

Q20. A `/map-services` skill reads hundreds of files and fills the session with output, so engineers lose earlier context. What's the most direct fix?

A. Add `allowed-tools: Read` to the frontmatter, so that it can only use read operations when it runs
B. Move the instructions into the root `CLAUDE.md`, so that they're always available to the main session
C. Tell engineers to run `/compact` immediately after every use of this skill, once it has finished running
D. Add `context: fork` to the skill's frontmatter, so that it runs isolated and doesn't fill the main session

Q21. An engineer wants a variation of the team's `/review` skill for personal experiments, without affecting anyone else. What should they do?

A. Create a personal skill in `~/.claude/skills/` with a different name
B. Add the personal variation as a section in the project's `CLAUDE.md`
C. Edit the team's skill locally, and take care never to commit the change
D. Create a `.claude/skills/review/` copy in a shared feature branch

Q22. Terraform conventions should apply whenever someone edits files under `infra/terraform/`, and test conventions for `*.test.tsx` files scattered around the frontend. What's the best configuration?

A. A `CLAUDE.md` in `infra/terraform/`, and another `CLAUDE.md` inside each directory containing test files
B. Two skills that engineers invoke by hand before they begin editing files of either of these kinds
C. A root `CLAUDE.md` with two headed sections, plus an instruction to apply only the relevant one
D. `.claude/rules/` files with `paths` globs, one for `infra/terraform/**/*` and one for `**/*.test.tsx`

Q23. Which task is the best fit for direct execution, without plan mode?

A. Splitting the billing module into two services, which requires decisions about their data boundaries
B. Adding a null check to one function, with a clear stack trace showing where the error occurs
C. Migrating a logging library used in 60 files, where two migration approaches are both possible
D. Choosing between Kafka and SQS for a new pipeline with different infrastructure requirements

Q24. An engineer must migrate an authentication library across 45 files, and has found that two approaches could work. What workflow is best?

A. Start with direct execution in one file, and see whether the same approach also works for the others
B. Stay in plan mode for the whole migration, because the size of the task makes every edit a planning matter
C. Use plan mode to explore and choose, then execute the agreed plan directly
D. Ask Claude to pick an approach by itself and complete it, then review the entire diff at the end

Q25. Claude's date-format conversions keep coming out wrong, despite two paragraphs describing the format. What's the most effective next step?

A. Restart the session, in case earlier context is causing the problem
B. Ask Claude to think harder about the date format before it writes
C. Add a third paragraph that describes the format in greater detail
D. Give 2–3 concrete input and output examples of the transformation

Q26. A review turned up three problems. A schema change affects both the data access code and the API response, while a typo in a log message is unrelated. How should the engineer send this feedback?

A. Send the two interacting schema issues together in one detailed message, and handle the typo separately
B. Fix the log message typo first, then send one schema issue, then wait to see if the other still happens
C. Send all three in a single message, because combining feedback is always faster and more efficient
D. Send each of the three issues in turn, so that Claude concentrates on only one thing at a time

Q27. An engineer is building a cache layer in a domain they don't know well, and they aren't sure what they've missed. Which technique does the guide recommend?

A. A long initial prompt listing every cache behavior that the engineer could find in textbooks
B. Direct execution, and then a review of whatever failure modes the first version happens to reveal
C. The interview pattern: have Claude ask questions that surface considerations before implementing
D. A batch of three independent implementations, of which the engineer then picks the best one

Q28. Midway through a long exploration, Claude refers to "the typical repository pattern" rather than the specific classes it found earlier, and gives inconsistent answers. What does this suggest, and what helps most?

A. Too few tools, so add more MCP servers that can search the repository in several different ways
B. Context degradation, so persist key findings to a scratchpad file and delegate verbose exploration to subagents
C. A faulty `CLAUDE.md`, so delete it and ask Claude to regenerate one from the current state of the code
D. A model outage, so wait a while and then repeat each of the same questions when the service has recovered

Q29. Context is filling with verbose discovery output in a session that's otherwise useful. Which command directly reduces context use?

A. `--resume`
B. `-p`
C. `/memory`
D. `/compact`

Q30. A long debugging session must pass a summary to the next phase. Which approach best preserves what the next phase needs?

A. Record specific identifiers, values and findings in a structured form, outside any free-form summary
B. Pass the complete raw tool output of every earlier step, so that nothing at all can possibly be lost
C. Rely on the model's memory, as it will remember what it found in earlier sessions automatically
D. Ask for a short natural-language summary, so that the next phase has as little context to carry as possible

Scenario: Multi-Agent Research System

Key idea

Meridian Insights runs a research system on the Agent SDK. A coordinator delegates to a web search subagent, a document analysis subagent, a synthesis subagent and a report subagent. Reports must be complete and cited.

Q31. A report on "the economics of offshore wind" covers only turbine costs. The logs show that the coordinator asked for only "offshore wind turbine costs", and that each subagent did its task well. Where's the problem?

A. The coordinator decomposed the topic too narrowly
B. The synthesis subagent dropped its other findings
C. The report subagent reduced the final page count
D. The web search subagent's queries were too broad

Q32. A simple factual question goes through search, analysis, synthesis and report in every case. What improves this?

A. Add a fifth subagent, whose only purpose is to classify query types for the coordinator
B. Remove the coordinator, and let the subagents decide among themselves who should handle a query
C. Have the coordinator analyze each query and invoke only the subagents that it requires
D. Run all four subagents in parallel, so that the fixed pipeline at least finishes more quickly

Q33. The search and analysis subagents keep returning overlapping material. What's the best fix?

A. Have the synthesis subagent remove the duplicates once the work has all been completed
B. Let them message each other, so that each can avoid what the other is already doing
C. Partition scope, giving each one distinct subtopics or source types
D. Reduce the number of results that each of the two subagents is allowed to return

Q34. After synthesis, the coordinator notices that a report has no data on regulation. What's the guide's approach?

A. Ask the synthesis subagent to write the regulation section from its own general knowledge of the topic
B. Restart the whole pipeline from the beginning, using the same task list as it used the first time
C. Re-delegate targeted queries to search and analysis, then re-invoke synthesis until coverage is sufficient
D. Publish the report as it stands, since the first synthesis is always the authoritative version of a report

Q35. The synthesis subagent writes reports that ignore the search results. Investigation shows the coordinator's prompt to it says only "synthesize the findings". What's the fix?

A. Include the complete findings from prior agents directly in the synthesis subagent's prompt
B. Give the synthesis subagent its own web search tool, so that it can repeat the same searches itself
C. Tell the synthesis subagent in its system prompt to remember the results from the search subagent
D. Give the synthesis subagent access to the coordinator's conversation history through a shared setting

Q36. Latency is high, because the coordinator starts four independent searches one after another. What's the correct way to run them in parallel?

A. Emit one Task tool call per turn, with instructions to hurry
B. Emit multiple Task tool calls in a single coordinator response
C. Move all four searches into one single large Task prompt
D. Use `fork_session` once for each of the search topics

Q37. A coordinator has `allowedTools: ["WebSearch", "Read"]`, and it has subagents defined, but they're never invoked. What's the most likely cause, by the guide?

A. The subagents need their own API keys
B. Only one subagent may be defined at a time
C. `end_turn` was returned by the first response
D. `"Task"` is missing from `allowedTools`

Q38. Findings pass between agents as paragraphs, and synthesized reports have lost their page numbers and URLs. What's the most effective fix?

A. Pass structured data that separates content from metadata such as URLs, document names and pages
B. Reduce the amount of text each subagent returns, so that the synthesis subagent's task is simpler
C. Ask the report subagent to search for each claim's source again, once the report text has been written
D. Instruct every subagent to take particular care to mention its sources at the end of each paragraph

Q39. Claude confuses `analyze_content` with `analyze_document`, whose descriptions are almost the same. In practice, the first handles web results. What's the guide's preferred fix?

A. Keep the names, and add to the system prompt that Claude should think carefully about which to use
B. Remove the description from both tools, so that Claude has to rely on the names and on context alone
C. Merge both of the tools into `analyze`, with a required `kind` parameter to tell them apart
D. Rename it `extract_web_results`, with a web-specific description that sets it apart

Q40. Tool descriptions are clear, but the coordinator picks the search tool whenever a message includes the word "find", even for local document work. What should you check?

A. The system prompt, for keyword-sensitive instructions that override the descriptions
B. The MCP server's connection timeout, as slow servers are more often ranked lower for selection
C. The number of tokens in `max_tokens`, since small limits affect which tool Claude selects
D. The JSON Schema `required` array, because optional inputs cause tools to be skipped

Q41. A search source times out. The subagent's retries fail, too. What should it send the coordinator?

A. Nothing at all, because subagents shouldn't report errors to the coordinator directly
B. The failure type, what it attempted, any partial results, and possible alternatives
C. A success status with an empty list, so that the rest of the workflow isn't interrupted
D. The message "search unavailable", with instructions to stop all work on the research

Q42. The synthesis subagent has started running its own web searches and returning poor results. What's the best fix?

A. Tell it in its system prompt never to use the web search tool
B. Remove web search from its tools, keeping its tool set focused on its role
C. Give it additional tools, so that it can use better sources
D. Lower its `max_tokens`, so that it has less room for searches

Q43. The synthesis subagent often needs to confirm one date or figure. Sending each one through the coordinator causes long delays. What's the guide's approach?

A. Accept the delays, because all information has to go through the coordinator without exception
B. Give it a scoped `verify_fact` tool, and route more complex cases through the coordinator
C. Give it the complete web search tool, so that it never needs to go back to the coordinator
D. Let it message the search subagent directly, and leave the coordinator out of these checks

Q44. Document analysis uses `fetch_url`, and sometimes fetches pages that aren't documents, including internal addresses. Which change does the guide recommend?

A. Replace it with a `load_document` tool that validates document URLs
B. Check the fetched pages afterwards, and remove anything that was not a document
C. Give the subagent a second fetch tool for use if the first tool's result fails
D. Add more detail to the `fetch_url` description about what it should not be used for

Q45. A report reads as complete, but a source had failed, and that subtopic has no findings. What should have happened?

A. Synthesis should annotate coverage, showing which areas are well supported and which have gaps from unavailable sources

B. The search subagent should have returned an empty success, so that the report could continue smoothly

C. The report subagent should have filled that subtopic from the other sections, to keep the report even

D. The whole workflow should have stopped, since a report with any failed source isn't useful to anyone

Scenario: Structured Data Extraction

Key idea

Ledgerline extracts data from supplier invoices and contracts. It uses Claude with JSON schemas to feed an accounting system. Documents vary widely. Some have no purchase order, and some have totals that don't match their line items.

Q46. Reviewers complain that the prompt "flag any suspicious invoice" gives inconsistent results. Which rewrite would improve this most?

A. Add "you're an experienced forensic accountant with decades of experience in invoices"
B. Add "be conservative, and flag only if you're highly confident that something is wrong"
C. Define specific conditions: for example, flag when line items don't sum to the total, or a supplier ID isn't on file
D. Ask the model to rate its confidence, then discard all flags below a chosen confidence level

Q47. Extraction works on tabular invoices but returns null for totals in narrative contracts, although they're present in the text. What's the most effective change?

A. Make the total field required, so that the model must return a value for every kind of document
B. Add a longer instruction saying that totals may appear anywhere in a document that's being read
C. Retry each null result with the same prompt, as a second pass may well find the missing value
D. Add few-shot examples showing correct extraction from documents with narrative and tabular formats

Q48. Which few-shot set would most improve judgment on ambiguous documents?

A. Several examples with correct output only, with all explanation removed in order to save on tokens
B. Twenty easy invoices of the same layout, to show the correct format as many times as possible
C. One complete example of a perfect invoice, along with an instruction to always copy its structure
D. Two to four hard cases, each showing the reasoning for its result and why a plausible alternative was rejected

Q49. Some invoices have no purchase order, yet extractions show plausible-looking PO numbers that appear nowhere in the document. What's the root-cause fix?

A. Add a post-processing step that deletes any PO number starting with an unusual prefix
B. Keep the field required, and add "don't invent values" to the system prompt in capitals
C. Make the field optional and nullable, so that the model isn't pressured to fill it
D. Raise the temperature, so that the model is less fixed on producing a plausible value

Q50. The schema's `expense_type` enum keeps failing on rare categories, and new ones appear every quarter. What's the best schema design? **Select 2.**

A. Remove the enum, and accept free text in all cases
B. Add an "unclear" value for ambiguous cases
C. Add an "other" value along with a detail string
D. Make the field required, with no other values added
E. List every expense type that has ever appeared in the data

Q51. Dates arrive as "3/4/26", "March 4, 2026" and "2026-03-04", all of which pass the schema as strings. What's the best addition?

A. A second schema for each of the different date formats in use
B. A larger number of retries for all documents containing dates
C. Removal of date fields from the schema, to be handled later
D. Format normalization rules in the prompt, alongside the strict schema

Q52. An extraction passes the schema, but its line items total 90 while `calculated_total` says 80. What's the best next step?

A. Send a follow-up with the document, the failed extraction and this specific error
B. Tighten the JSON schema's number types so that this sum error can't be expressed
C. Accept the output, because a schema-valid result has been shown to be correct
D. Reissue the same request with no changes, as a new attempt may get a better result

Q53. A contract's renewal date is only in an annex that wasn't provided, and validation keeps failing on it. Choose the correct analysis. **Select 2.**

A. A higher effort setting will allow the model to infer the date
B. The date should be estimated from the contract's start date
C. More retries won't help, as the data isn't in the source
D. More specific error feedback will allow the model to find the date
E. Null or routing for review is more appropriate than retrying

Q54. An invoice states a total of 95, and its items add up to 90. The extraction read both correctly. Which schema supports the best handling?

A. Both `stated_total` and `calculated_total`, plus a `conflict_detected` boolean for review
B. A single `total` field, into which the model places the number that it trusts the most
C. A `calculated_total` only, because a computed value should always replace the printed one
D. A `stated_total` only, because a printed value is the legal record of the transaction

Q55. Ledgerline has a nightly job for 40,000 invoices, due by morning, and a live "extract now" button for users. How should these run?

A. Synchronous API for the job, and batch for the button, since users can wait a few minutes
B. Synchronous API for both, as batch processing can't be used with JSON schemas or tools
C. Batch API for the nightly job, and the synchronous API for the button
D. Batch API for both, since the 50% saving applies equally to each of these workflows

Q56. Results must arrive within 30 hours, and batches take up to 24. Which submission plan guarantees that?

A. Submit one batch every 24 hours
B. Submit when 10,000 documents accumulate
C. Submit a batch at least every 4 hours
D. Submit one batch every 12 hours

Q57. A batch of 2,000 returns 60 failures, all with documents exceeding context limits. What's the efficient response?

A. Resubmit only those 60, identified by `custom_id`, with each one split into chunks
B. Resubmit the 60 exactly as they were, as batch failures are usually only temporary
C. Resubmit the whole batch of 2,000 with a shorter prompt, so that they all succeed together
D. Send the 60 to a human, because documents over the limit can't be processed by Claude

Q58. Field accuracy is 97% overall, and leadership wants to remove human review. What should happen first?

A. Ask the model to report its own accuracy on the documents, and compare it to 97%
B. Remove review at once, because 97% exceeds the accuracy of most human reviewers
C. Lower the review rate gradually, without segment analysis, and watch for complaints
D. Validate accuracy by document type and field, since the aggregate can hide weak segments

Q59. High-confidence extractions now skip review. How do you keep measuring their error rate and detect new error patterns?

A. Review a stratified random sample of them on an ongoing basis
B. Stop measuring once the error rate has stayed low for any full quarter of production work
C. Review the first hundred documents of each month, in the order in which they happen to arrive
D. Review only low-confidence extractions, as high-confidence ones have been established as correct

Q60. Reviewers can only handle 5% of documents. Which ones should they see, by the guide?

A. A purely random 5%, so that no group of documents ever gets any special priority
B. Those with low model confidence, or ambiguous or contradictory source documents
C. The most recent documents, as older ones will already have been corrected elsewhere
D. Those from the largest suppliers, as they account for the greatest amount of money

Answer Key and Score Sheet

Q	Ans	Q	Ans	Q	Ans	Q	Ans
1	C	2	D	3	B	4	B
5	A	6	C	7	D	8	C
9	B	10	B	11	B, E	12	D
13	B	14	D	15	B	16	B
17	C	18	B, D	19	C	20	D
21	A	22	D	23	B	24	C
25	D	26	A	27	C	28	B
29	D	30	A	31	A	32	C
33	C	34	C	35	A	36	B
37	D	38	A	39	D	40	A
41	B	42	B	43	B	44	A
45	A	46	C	47	D	48	D
49	C	50	B, C	51	D	52	A
53	C, E	54	A	55	C	56	C
57	A	58	D	59	A	60	B

Domain	Items	Your score	Target
1. Agentic Architecture and Orchestration	16		70%+
2. Tool Design and MCP Integration	11		70%+
3. Claude Code Configuration and Workflows	12		70%+
4. Prompt Engineering and Structured Output	12		70%+
5. Context Management and Reliability	9		70%+
Total	**60**		**80%+**

The guide doesn't publish how raw scores map to the scaled 720, so these targets are only study guidance.

Your error log: item, what you picked, why it was wrong

Explanations

1. C (TS 1.1). A response can carry text alongside a tool call, so text doesn't mean completion. `stop_reason` is the explicit signal. The prompt rule and the regex keep a fragile signal, and a fixed count is the guide's named anti-pattern when used as the main stop.

2. D (TS 1.1). High-ambiguity requests suit model-driven decisions, while guarantees belong in code. A decision tree or a fixed sequence can't adapt, and an instruction alone is probabilistic.

3. B (TS 1.4). Any prompt technique leaves a non-zero failure rate. A programmatic prerequisite makes the wrong order impossible. A text confirmation is still model output, and not a check.

4. B (TS 1.4). The guide's pattern is decompose, investigate using shared context, then synthesize. Handling only the first drops work, multi-concern isn't an escalation trigger, and separate sessions lose shared facts.

5. A (TS 1.4). Humans lack the transcript, so the handoff has to carry the decision-ready facts. A pointer assumes access they don't have, one line forces re-investigation, and a raw dump buries what they need.

6. C (TS 1.5). PostToolUse transforms results deterministically before the model reads them. A prompt stays probabilistic, a subagent adds cost and another model step, and the arguments don't control these backends' formats.

7. D (TS 1.5). The hook guarantees the limit and redirects the work. Removing the tool breaks first-contact resolution, a silent cap misleads the customer, and prompts don't guarantee compliance.

8. C (TS 1.6). Chains fit predictable work, and returns and disputes vary case by case. Reviewability doesn't override fit, nothing shows resolution would improve, and chains can call tools.

9. B (TS 2.1). Descriptions are the primary selection mechanism, so the root-cause fix is also the simplest one. A classifier overbuilds, merging hides the distinction, and few-shot examples work around weak descriptions.

10. B (TS 2.2). Structured metadata lets the agent choose a recovery. Empty results falsely signal no orders, a bare status gives no recovery information, and free text can't be relied on for the decision.

11. B and E (TS 2.2). It's a business error: not retryable, and worth explaining. Calling it transient wastes retries, a generic message prevents a proper explanation, and a lower amount doesn't change the expired window.

12. D (TS 2.3). Only the forced form names the tool, per the guide. `any` lets Claude choose which, `auto` may return text, and array order isn't a guarantee. Build today: forced forms are rejected on some current models.

13. B (TS 2.3). The guide's reason is tool selection reliability. There's no such fixed MCP limit, no automatic rejection as stated, and no requirement for one subagent per tool.

14. D (TS 5.1). Progressive summarization loses transactional details, so they live outside it. More frequent summaries lose more, `max_tokens` doesn't change what's preserved, and an instruction isn't dependable.

15. B (TS 5.2). Sentiment doesn't measure complexity, and the issue is within the agent's capability. The guide says to acknowledge and resolve, and to escalate if they reiterate a preference for a human.

16. B (TS 3.1). User-level config isn't shared. Project scope and version control are the fix. The other options keep the manual copying that caused the problem.

17. C (TS 3.1). `/memory` shows what's loaded, which is the first thing to compare. Reinstalling doesn't address config, self-description isn't evidence, and deleting everything is far too disruptive for a first step.

18. B and D (TS 3.1). Rules files and selective imports are the guide's modular options. User-level loses sharing, asking Claude to skip still loads everything, and an MCP tool isn't how the guide organizes standing instructions.

19. C (TS 3.2). Project-scoped commands are shared by version control. User-level ones are personal, `CLAUDE.md` isn't for commands, and an MCP entry is unrelated. Build today: a skill at `.claude/skills/changelog/SKILL.md` also creates `/changelog`.

20. D (TS 3.2). Isolation is the purpose of `context: fork`. Compacting afterwards still loses context, restricting tools doesn't reduce the read output, and moving it to `CLAUDE.md` adds always-on load.

21. A (TS 3.2). Personal variants with a different name avoid affecting teammates. A local edit risks accidental commits, a shared branch affects others, and `CLAUDE.md` changes are team-wide.

22. D (TS 3.3). Path-scoped rules load only for matching files, and globs handle scattered files. Per-directory files don't scale for spread files, a root file always loads, and manual skills are easy to forget.

23. B (TS 3.4). A clear, small scope suits direct execution. The other three involve architecture, many files or several valid approaches.

24. C (TS 3.4). The guide combines planning for investigation with direct execution for implementation. Starting directly risks costly rework, plan mode doesn't make edits, and a blind choice skips the decision.

25. D (TS 3.5). Examples are the guide's best tool when prose gets interpreted inconsistently. More prose repeats what failed, and neither thinking harder nor a restart addresses the ambiguity.

26. A (TS 3.5). Interacting problems belong together, while independent ones can be handled alone. Splitting interacting fixes risks conflicts, and "always" ignores the independent case.

27. C (TS 3.5). The interview pattern surfaces things like invalidation and failure modes that the developer hadn't anticipated. The other options find them late, or depend on what the engineer already knows.

28. B (TS 5.4). These are the guide's symptoms of degradation in extended sessions. Scratchpads and subagents are its remedies. The other diagnoses don't fit the symptoms.

29. D (TS 5.4). `/compact` reduces context use. `/memory` shows loaded memory files, `-p` is non-interactive mode, and `--resume` continues a session.

30. A (TS 5.1). Specifics are what summaries lose, so keep them structured. A short summary loses them, raw output overwhelms the context, and models have no automatic memory across sessions.

31. A (TS 1.2). Subagents did what they were asked, so the narrow task list caused the gap. This is the guide's named risk of overly narrow decomposition.

32. C (TS 1.2). Dynamic selection by query complexity is a coordinator duty in the guide. Removing it loses control, parallelism keeps the waste, and a new subagent overbuilds a coordinator's task.

33. C (TS 1.2). Partitioning prevents the duplication at the source. Direct messaging breaks hub-and-spoke routing, deduplicating afterwards still pays for the wasted work, and lower limits lose good results as well.

34. C (TS 1.2). The guide describes this iterative refinement loop. Publishing hides the gap, general knowledge loses citations, and an identical restart repeats the same gap.

35. A (TS 1.3). Subagents don't inherit context, so it must be passed in. No inherited history is available as described, it has nothing to remember, and repeating searches wastes work and strays from its role.

36. B (TS 1.3). The guide says multiple Task calls in one response. Separate turns are sequential, `fork_session` branches a session's history and doesn't run subagents, and one large prompt isn't parallel.

37. D (TS 1.3). The guide requires `"Task"` in the coordinator's `allowedTools`. Build today: the current name is `Agent`. No separate keys are needed, several subagents are allowed, and the last option describes a symptom at most.

38. A (TS 1.3). Structure stops attribution being lost in prose. An instruction is unreliable, re-searching may attach wrong sources, and shorter text does nothing to preserve metadata.

39. D (TS 2.1). Renaming and redescribing removes the overlap. Removing descriptions worsens selection, a prompt doesn't resolve overlap, and merging hides the choice behind a parameter.

40. A (TS 2.1). Keyword-sensitive wording in a system prompt can create unintended tool associations. The others aren't connected with word-triggered selection.

41. B (TS 2.2). Structured context lets the coordinator decide what to do next. Empty success hides the failure, a generic status drops what's useful, and silence leaves the coordinator unaware.

42. B (TS 2.3). Scoped access prevents cross-specialization misuse at the source. A prompt is only guidance, a lower `max_tokens` is unrelated, and more tools make the problem worse.

43. B (TS 2.3). A limited cross-role tool handles the high-frequency need. The full tool invites misuse, direct messaging breaks the topology, and the guide allows scoped cross-role tools.

44. A (TS 2.3). A constrained tool enforces the boundary. A description is only guidance, a second tool adds nothing, and checking afterwards happens too late.

45. A (TS 5.3). Coverage annotations are the guide's answer. Stopping and silently suppressing are both named anti-patterns, and filling in fabricates coverage.

46. C (TS 4.1). Specific criteria beat vague or confidence-based filtering. A persona doesn't define what's suspicious, and self-reported confidence isn't a reliable filter.

47. D (TS 4.2). Few-shot examples address empty extraction from varied formats. Required fields encourage invention when a value is truly absent, and instructions and bare retries are weaker fixes.

48. D (TS 4.2). The guide recommends 2-4 targeted examples that show reasoning for ambiguous cases. Easy repetitions don't teach judgment, one example can't cover variation, and removing reasoning drops the teaching.

49. C (TS 4.3). Required fields pressure the model to fabricate. Nullable removes that pressure. Post-processing can't detect plausible fakes, an instruction leaves the pressure in place, and temperature doesn't address it.

50. B and C (TS 4.3). "Other" plus detail handles new categories, and "unclear" handles ambiguity. A giant enum still misses new ones, free text loses structure, and a required closed enum forces wrong fits.

51. D (TS 4.3). The guide pairs normalization rules with the schema, since schemas alone don't fix inconsistent formats. Multiple schemas add complexity, retries don't help valid-looking output, and removing fields loses data.

52. A (TS 4.4). This is a semantic error, and specific feedback lets the model correct itself. A bare retry has no new information, schemas can't express the sum rule here, and valid isn't the same as correct.

53. C and E (TS 4.4). Retries can't recover absent data. Better feedback and more effort don't create it, and estimating is fabrication.

54. A (TS 4.4). Keeping both and flagging preserves the evidence, so a person or rule can decide. Each single-value schema discards information about the discrepancy.

55. C (TS 4.5). Match tempo to need. Batch has no latency guarantee, so it's wrong for a live button. Batches can use tools, just not multi-turn tool loops within a request. **Check that a morning deadline allows for a 24-hour worst case.**

56. C (TS 4.5). Worst case is window plus 24 hours, so 4 + 24 = 28 fits, while 12 + 24 = 36 and 24 + 24 = 48 don't. A count-based trigger gives no time guarantee.

57. A (TS 4.5). `custom_id` isolates failures, and chunking addresses the actual cause. Rerunning everything wastes money, an identical retry fails again, and chunking makes humans unnecessary here.

58. D (TS 5.5). Aggregates can mask poor performance in specific types or fields. Waiting for complaints detects errors too late, and self-reporting doesn't measure accuracy. There's also no evidence for the human comparison.

59. A (TS 5.5). Stratified sampling of high-confidence output measures the error rate and catches novel patterns. The others leave blind spots, or stop looking.

60. B (TS 5.5). Limited capacity goes to the likeliest problems. A random sample has its own use in measurement, but this asks about prioritizing attention. Supplier size and recency aren't the guide's criteria.